



AfriOS

Modules

Référence par sous-système

Description détaillée de chaque module d'AfriOS : firmware, noyau, orchestrateur, 5 couches de compatibilité, modules support, build et tests. 12 modules, ~250 fichiers, ~44 000 lignes.

VERSION

1.0

DATE

18 August 2026

AUTEUR

Équipe AfriOS

Document technique AfriOS — github.com/ErdisKodjo/System

STATUT

Public

Table des matières

Table des matières	2
1. Vue d'ensemble des modules	4
1.1 Tableau récapitulatif	4
1.2 Cartographie des dépendances	4
2. Module FirmwareHybride	5
2.1 Structure interne	5
2.2 Fichiers principaux	5
2.3 API exposée	6
3. Module afros-core	7
3.1 HAL — Hardware Abstraction Layer	7
3.2 Ports HAL	7
3.3 Sous-systèmes noyau (afros/)	7
3.4 API publique noyau	8
4. Module afros-corebridge-core	9
4.1 Structure interne	9
4.2 API publique (gelée v1.0.0)	9
4.3 Exemple d'usage	9
5. Module afros-winbridge	11
5.1 Sous-systèmes	11
5.2 Fichiers clés	11
6. Module afros-androsandbox	12
6.1 Sous-systèmes	12
6.2 Binder IPC	12
6.3 ART runtime	12
7. Module afros-incompat-engine	13
7.1 Sous-systèmes	13
7.2 Runtime Objective-C	13
7.3 Code signing	13
8. Module afros-harmonygate	14
8.1 Sous-systèmes	14
8.2 SoftBus — bus distribué	14
8.3 Ability runtime	14

9. Module afros-dxvk	15
9.1 Sous-systèmes	15
9.2 Points d'entrée Wine	15
9.3 Auto-déclaration Vulkan	15
10. Modules support	16
10.1 afros-network	16
10.2 afros-storage	16
10.3 afros-power-management	16
11. Build, tests et CI	17
11.1 Build system	17
11.2 Tests	17
11.3 CI/CD	17
11.4 Scripts d'automatisation	17
12. Annexe — Index des fichiers clés	18
13. Conclusion	18

1. Vue d'ensemble des modules

Ce document sert de référence technique par sous-système. Pour chacun des 12 modules qui composent AfriOS, il présente la structure interne, les fichiers principaux, l'API exposée, et les dépendances. Ce document est conçu pour être consulté non-linéairement : chaque section est autonome et peut être lue indépendamment.

1.1 Tableau récapitulatif

Module	Lignes	Fichiers	Rôle
FirmwareHybride	~6 400	~40	Firmware UEFI/EDK2 multi-arch
afros-core	~3 000	79	Noyau + HAL + pilotes
afros-corebridge-core	5 139	27	Orchestrateur multi-runtime
afros-dxvk	5 542	27	DirectX → Vulkan
afros-winbridge	6 160	40	Compatibilité Windows (Wine)
afros-androsandbox	7 332	42	Compatibilité Android
afros-incompat-engine	6 978	42	Compatibilité iOS/macOS
afros-harmonygate	5 172	26	Compatibilité HarmonyOS
afros-network	~300	6	Stack réseau intelligent
afros-storage	~250	5	Gestion stockage (AfrosFS)
afros-power-management	~300	5	Gestion énergie
Tests + CI + scripts	~3 500	~80	Tests, build, automatisation
Total	~50 000	~420	

1.2 Cartographie des dépendances



2. Module FirmwareHybride

Le firmware AfriOS est une platform package EDK2 (TianoCore) qui implémente le boot UEFI multi-architectures avec extensions hybrides : secure boot, measured boot, OTA A/B, hyperviseur minimal. Il supporte x86_64, AARCH64 et RISC-V 64.

2.1 Structure interne

```
FirmwareHybride/  +-- edk2/HybridFirmwarePlatformPkg/  +-- BootManager/  |  +--
UefiBootManager/  LinuxBootManager.c - GenericBootManager.c  |  +-- WindowsBoot/
WinBootMgr.c  |  +-- AppleBoot/  AppleBootHelper.c - ConfigPlistParser.c  |  +--
PxeBoot/  IpxeDriver.c - HttpBootClient.c  |  +-- BootMenu/  BootMenuUi.c -
BootScripts.c  |  +-- LegacyCsm/  CsmPolicy.c  |  +-- BootPolicyEngine.c  +-- OtaUpdate/
|  +-- ABSlotManager.c  |  +-- FwUpdateAgent.c  |  +-- CapsuleEngine.c  +-- Diagnostics/
|  +-- PowerOnSelfTest/  MemoryTest.c - PciEnumTest.c  |  +-- UefiShell/
ShellExtensions.c - DebugScripts.nsh  +-- Security/  |  +-- SecureBoot/
SecureBootPolicy.c  |  +-- MeasuredBoot/  MeasuredBootLib.c  +-- ShimLayer/  |  +--
ShimLoader.c  |  +-- MinimalHypervisor/  VmcsInit.c - Passthrough.c  +-- PlatformInit/
|  +-- Sec/  MultiArchResetVector.S  |  +-- Pei/  MemoryInit.c - PlatformInfoPei.c  |
+-- Dxe/  PlatformInitDxe.c - AcpiPlatformDxe.c  +-- HardwareAbstractionLayer/  |  +--
CpuHal/  CpuHalLib.c  |  +-- PlatformDetect/  PlatformDetectLib.c  |  +-- Acpi/
AcpiTableGenerator.c - Fadt.c - Dsdt.asl  |  +-- Smbios/  SmbiosGenerator.c  |  +--
DeviceTree/  FdtPlatformDxe.c  +-- SetupUi/HiiForms/  MainForm.vfr -
BootOrderForm.vfr + .uni  +-- Libraries/TimerLib/  TimerLib.c  +-- BoardConfigs/  rpi4
/  pine64 /  qemu-virt /  qemu-virt-riscv  +-- HybridFirmwarePlatformPkg.dsc /  .dec
```

2.2 Fichiers principaux

Fichier	Lignes	Rôle
WinBootMgr.c	304	Boot Windows via bootmgfw.efi
ConfigPlistParser.c	584	Parseur XML config.plist (OpenCore/Clover)
AppleBootHelper.c	422	Boot macOS via boot.efi + boot args
IpxeDriver.c	317	Driver binding iPXE pour boot réseau
HttpBootClient.c	357	Client HTTP Boot (DHCP opt 60 + HTTP GET)
BootMenuUi.c	268	Menu texte multi-OS avec navigation clavier
BootScripts.c	398	Moteur de scripts .nsh-like
ABSlotManager.c	362	Gestion slots A/B avec rollback NVRAM
FwUpdateAgent.c	464	Agent mise à jour capsule UEFI
MemoryTest.c	275	POST mémoire (pattern 0x55AA55AA)
PciEnumTest.c	224	Énumération PCI via EFI_PCI_IO_PROTOCOL
ShellExtensions.c	516	Commande afri (ver/slot/capsule/pci/memtest)
Passthrough.c	353	Passthrough EPT/Stage-2 1:1 hyperviseur
Dsdt.asl	185	DSDT ACPI (CPU, HPET, RTC, device AfriOS)

Fichier	Lignes	Rôle
BootOrderForm.vfr	171	Formulaire VFR boot order + secure boot

2.3 API exposée

Le firmware expose son API via des protocols UEFI standards (EFI_BOOT_MANAGER_LOAD_OPTION, EFI_HTTP_BOOT_CALLBACK_PROTOCOL, EFI_SHELL_DYNAMIC_COMMAND_PROTOCOL) et des variables NVRAM spécifiques AfriOS (AfriBootInfo, AfriCsmPolicy, AfriFwUpdateState).

3. Module afros-core

Le module afros-core contient le noyau AfriOS générique et la couche d'abstraction matérielle (HAL). Il est structuré en 4 sous-répertoires : hal/ (interfaces abstraites), ports/ (implémentations par architecture), afros/ (logique noyau générique), drivers/ (pilotes périphériques).

3.1 HAL — Hardware Abstraction Layer

La HAL définit 6 interfaces abstraites via des op-tables C (structs de pointeurs de fonctions). Chaque port fournit une implémentation concrète de ces tables.

Op-table	Header	Fonctions clés
arch_cpu_ops	cpu_abstraction.h	init, get_info, set_frequency, sleep/wakeup, migrate_task
arch_memory_ops	memory_abstraction.h	alloc, free, map, unmap, compress, decompress
arch_interrupt_ops	interrupt_abstraction.h	init, enable, disable, register_handler, ack, send_ipi
arch_timer_ops	timer_abstraction.h	init, get_ticks, set_one-shot, set_periodic, busy_wait_us
arch_console_ops	console_abstraction.h	init, putc, puts, getc, has_input
arch_storage_ops	storage_abstraction.h	init, get_info, read_blocks, write_blocks, flush

3.2 Ports HAL

Cinq ports HAL sont disponibles. Chacun implémente les 6 op-tables avec les spécificités de son architecture.

Port	Arch	Console	Timer	Spécificités
port-arm64	AArch64	PL011 UART	cntpct_el0	big.LITTLE, GIC v3
port-x86_64	x86_64	VGA + 16550	RDTSC	APIC, IOAPIC
port-riscv	RISC-V 64	SBI console	rdtime	S-mode, PLIC
port-mcu	Cortex-M	USART STM32	SysTick	NVIC, mono-cœur
port-host-mock	Linux host	printf	clock_gettime	tests CI sans QEMU

3.3 Sous-systèmes noyau (afros/)

Le noyau générique est organisé en 6 sous-systèmes :

- scheduler/ — afros_cfs.c (CFS), big_little.c + big_little_support.c (placement big.LITTLE), power_aware.c + power_aware_sched.c (ordonnancement power-aware), task_migration.c.

- memory/ — adaptive_reclaim.c (récupération adaptative), compression.c (compression pages), adaptive_swap.c, numa_aware.c + numa_balancer.c (NUMA balancing multi-socket).
- network/ — tcp_westwood_africa.c (TCP Westwood+ Africa), intermittent_net.c (DTN-lite), protocol_compression.c (ROHC), tcp_acceleration.c, intelligent_routing.c, protocol_optimization.c.
- power/ — solar_aware.c (gestion solaire), opportunistic_compute.c (calcul opportuniste), opportunistic_sleep.c, brownout_protection.c.
- fs/afrosfs/ — journal.c (journaling), file.c (opérations fichier).
- security/ — secure_boot.c (vérification boot).

3.4 API publique noyau

```
// Entry point void kernel_main(void); // HAL ops (fournies par le port actif)
extern hal_ops_t afros_hal_ops; extern cpu_ops_t arch_cpu_ops; extern memory_ops_t
arch_memory_ops; extern interrupt_ops_t arch_interrupt_ops; extern timer_ops_t
arch_timer_ops; extern console_ops_t arch_console_ops; extern storage_ops_t
arch_storage_ops; // Freestanding printf int kprintf(const char *fmt, ...); //
Logging #define afros_log_info(fmt, ...) kprintf("[INFO ] " fmt, ##__VA_ARGS__)
#define afros_log_warning(fmt, ...) kprintf("[WARN ] " fmt, ##__VA_ARGS__) #define
afros_log_error(fmt, ...) kprintf("[ERROR] " fmt, ##__VA_ARGS__)
```


4. Module afros-corebridge-core

L'orchestrateur central d'AfriOS. Détecte les formats exécutables, sélectionne le runtime approprié, fournit une vue unifiée du système (filesystem, address space, network, resources) à travers les 5 runtimes de compatibilité.

4.1 Structure interne

```
afros-corebridge-core/ +- include/ | +- afros_corebridge.h (umbrella header,
v1.0.0) | +- loader.h (app_type_t, format_info_t, runtime_handle_t) | +-
runtime_manager.h (runtime_ops_t vtable) | +- version_mgmt.h (version_t, quota_t,
usage_t) | +- orchestrator.h (entry points) | +- babelbridge.h +- loader/ | +-
app_detector.c (magic bytes PE/ELF/Mach-O/DEX/HAP) | +- format_analyzer.c
(sous-système PE/ELF arch, etc.) | +- intelligent_loader.c (décision + cache LRU)
| +- dependency_resolver.c (imports/DT_NEEDED/LC_LOAD_DYLIB) +-
runtime_managers/ | +- linux_runtime_manager.c (clone + CLONE_NEWNS|NEWPID) | +-
win_runtime_manager.c (wineserver + wine_loader) | +- android_runtime_manager.cpp
(service_manager + dalvikvm) | +- ios_runtime_manager.cpp (dyld_emulator +
objc_runtime) | +- harmony_runtime_manager.c (ability_runtime + HAP install) +-
unified_execution/ | +- filesystem_view.c (VFS union + path translation) | +-
address_space.c (mmap registry cross-runtime) | +- network_stack.c (netns + NAT +
port forwarding) | +- resource_manager.c (quotas CPU/mem/IO/FD/ports) +-
version_management/ | +- version_registry.c (JSON registry
/var/lib/afros/runtimes.json) | +- update_checker.c (manifest distant + semver
compare) | +- downloader.c (curl/wget + SHA-256) | +- installer.c (extraction +
self-test + rollback) +- src/ | +- api_version.c
(orchestrator_init/run_app/shutdown) | +- central_manager.c
(orchestrator_monitor_system) | +- selection_engine.c (compat score + budget) |
+- monitoring.c (watchdog 5s + /proc/pid/statm) | +- resource_allocator.c (pool
256 MiB) | +- runtime_registry.c (table runtimes actifs) +- rfcs/
(RFC-PROCESS.md + 2 RFCs) +- API.md / CHANGELOG.md
```

4.2 API publique (gelée v1.0.0)

L'API est organisée en 3 stability tiers : Tier 1 Stable (breaking $\Rightarrow$ MAJOR + RFC), Tier 2 Beta (peut évoluer entre minor), Tier 3 Experimental (instable).

Tier	API	Fonctions clés
1 Stable	Loader	AppDetect, FormatAnalyze, IntelligentLoad, ResolveDeps
1 Stable	Runtime Manager	LinuxRuntimeInit, WinRuntimeInit, etc.
1 Stable	Unified Execution	VfsCreateView, AsReserve, NetCreateNamespace, ResSetQuota
1 Stable	Orchestrator	orchestrator_init, orchestrator_run_app, orchestrator_shutdown
2 Beta	Version Mgmt	VersionRegister, UpdateCheck, DownloaderFetch, InstallerInstall
2 Beta	Selection/Monitor	SelectRuntime, MonitorStart, MonitorGetStats
3 Experm.	—	Aucune API publique en v1.0.0

4.3 Exemple d'usage

```
#include "afros_corebridge.h" int main(int argc, char **argv) { if (argc < 2)
return 1; // Initialiser l'orchestrateur if (orchestrator_init() !=
AFROS_CB_SUCCESS) { fprintf(stderr, "init failed\n"); return 1; } // Lancer l'app
(détection auto du format) int rc = orchestrator_run_app(argv[1], argv[2]); if (rc
!= AFROS_CB_SUCCESS) { fprintf(stderr, "run failed: %d\n", rc); }
orchestrator_shutdown(); return rc; }
```

5. Module afros-winbridge

Couche de compatibilité Windows basée sur Wine. Implémente un PE loader, un traducteur de syscalls NT → Linux, un émulateur de registry avec 5 hives binaires, un serveur COM, et un wineserver userspace.

5.1 Sous-systèmes

Sous-système	Fichiers	Rôle
pe_loader/	4	Parser PE/COFF, mapping mémoire, relocations, ressources
filesystem/	4	Path translator, drive manager, NTFS emulation
registry/	6	5 hives (HKCR/HKCU/HKLM/HKU/HKCC), hive_io binaire
syscall/	4	Traducteur NT → Linux, IRP queue, kernel objects
server/	3	wineserver Unix socket, process table, registry server
services/	4	SCM, MSI installer, Event Log, Task Scheduler
com/	5	CoInitialize, marshaler, proxy/stub, TypeLib, ActiveX
loader/	2	ELF preloader, wine_loader main
cache/	4	DLL cache, shader cache, JIT cache, registry cache

5.2 Fichiers clés

- pe_parser.c — Parse PE/COFF : DOS header, NT headers, sections, imports/exports. Expose PeLoadFromFile(), PeGetEntryPoint().
- syscall_translator.c — Traduit 15 NT syscalls (NtCreateFile, NtReadFile, NtWriteFile, NtClose, NtAllocateVirtualMemory, etc.) en syscalls Linux.
- registry_emulator.c — API Win32 registry (RegOpenKey, RegQueryValue, RegSetValue) qui dispatch vers hive_manager.
- hive_io.c — Lecteur/écriture du format binaire hive Windows (cells, bins, hive bin header).
- wineserver.c — Serveur principal : socket listener, client registry, request dispatcher, SIGCHLD reaper.

6. Module afros-androsandbox

Couche de compatibilité Android. Implémente le binder IPC, le runtime ART (JIT + class linker), SurfaceFlinger (compositeur), et les services framework Android (ActivityManager, WindowManager, PackageManager, etc.).

6.1 Sous-systèmes

Sous-système	Fichiers	Rôle
binder/	5	binder_driver (kernel-side), parcel, transaction, service_manager
dalvikvm/	1	CLI entry pour VM Dalvik
art/runtime/	3	ArtRuntime singleton, ClassLinker, JIT compiler
dex2oat/	2	AOT compiler (DEX → OAT/ELF)
compiler/	2	Optimizing compiler + dex_to_native fast path
surfaceflinger/	4	Composer 60 Hz, Layer, BufferQueue, HWC HAL
framework/activity_manager/	3	AMS, ActivityStack, TaskRecord
framework/window_manager/	3	WMS, DisplayContent, WindowState
framework/package_manager/	3	PMS, PackageParser (binary XML), PermissionManager
framework/content/	3	ContentProvider, ContentResolver, UriMatcher
framework/telephony/	2	TelephonyManager, PhoneInterface
services/	5	Camera, Audio, Sensor, Notification, Location
vfs/	5	AndroidFs, SdcardEmulation, Ashmem, Properties, SELinux

6.2 Binder IPC

Le binder est le bus IPC central d'Android. AfriOS implémente un binder_driver simulé en userspace qui gère une transaction queue, un thread pool, et des death notifications. Le service_manager expose 15 services par défaut (activity, package, window, etc.).

6.3 ART runtime

Le runtime ART (Android Runtime) est composé de 3 briques : art_runtime.cc singleton qui démarre heap + GC thread + JIT, class_linker.cc qui résout les classes depuis DEX et maintient une class table, jit_complier.cc qui compile les méthodes chaudes en code natif avec OSR (On-Stack Replacement).

7. Module afros-incompat-engine

Couche de compatibilité iOS/macOS basée sur Darling. Implémente un loader Mach-O, le runtime Objective-C (objc_msgSend, ARC), les frameworks UIKit/Foundation/CoreAnimation/CoreGraphics, la vérification de code signing, et le sandbox iOS (containers + entitlements).

7.1 Sous-systèmes

Sous-système	Fichiers	Rôle
macho_loader/	5	Parser Mach-O, loader, symbol_resolver, binding_handler, dyld_emulator
runtime/	4	objc_runtime, class_loader, message_dispatch, arc_implementation
filesystem/	4	Bundle manager, APFS emulation, HFS+ emulation, iCloud stub
sandbox/	4	iOS sandbox, container_manager, entitlements, data_protection
codesign/	3	signature_verifier, certificate_chain, provisioning_profile
frameworks/UIKIT/	5	UIApplication, UIView, UIWindow, UIViewController, UIControl
frameworks/Foundation/	5	NSObject, NSString, NSArray, NSDictionary, NSData
frameworks/AVFoundation/	3	AVPlayer, AVAsset, AVAudioSession
frameworks/CoreAnimation/	3	CALayer, CAAnimation, CATransaction
frameworks/CoreGraphics/	4	CGContext, CGPath, CGColor, CGImage
darling/src/	2	startup, darling_kernel (mach_port_t, mach_msg)

7.2 Runtime Objective-C

Le runtime Objective-C est la pièce centrale. objc_runtime.c gère l'enregistrement des classes, le dispatch de méthodes, le layout des ivars, et la fusion des categories. message_dispatch.c implémente objc_msgSend avec taggable pointer classes, IMP cache, et super dispatch. arc_implementation.c fournit objc_retain, objc_release, objc_autorelease, et weak refs.

7.3 Code signing

Trois modules vérifient l'authenticité des apps iOS : signature_verifier.c valide la signature CMS du binaire Mach-O, certificate_chain.c construit et vérifie la chaîne jusqu'à Apple Root CA, provisioning_profile.c parse les profiles .mobileprovision (CMS-wrapped plist) pour extraire l'app ID et le team ID.

8. Module afros-harmonygate

Couche de compatibilité HarmonyOS. Implémente le SoftBus (bus IPC distribué multi-transport), la sync de données distribuée avec résolution de conflits, le sharing matériel cross-device, et l'ability runtime (framework applicatif HarmonyOS).

8.1 Sous-systèmes

Sous-système	Fichiers	Rôle
distributed/device_manager/	4	Discovery (mDNS), monitor, capability, trust (PIN pairing)
distributed/data_sync/	4	DistributedDataMgr, sync_engine, versioning (vector clocks), conflict_resolution/
distributed/softbus/discovery/	3	BLE, mDNS, CoAP discovery
distributed/softbus/connection/	3	TCP, Bluetooth, Wi-Fi Direct transports
distributed/softbus/authentication/		HMAC challenge-response + session keys
distributed/softbus/transmission/	2	File transmission (chunked), stream (low-latency)
distributed/hardware_sharing/	3	Camera, storage, sensor sharing cross-device
ability/ability_runtime/	3	Ability lifecycle, AbilityManager, AbilityContext
ability/want/	2	Want (action/entity/uri), WantParams (key-value bundle)

8.2 SoftBus — bus distribué

Le SoftBus est la pierre angulaire de HarmonyOS distributed. Il supporte 3 mécanismes de discovery (BLE, mDNS, CoAP), 3 transports (TCP, Bluetooth, Wi-Fi Direct), une authentification HMAC challenge-response avec clés de session, et 2 modes de transmission (file chunked + stream low-latency).

8.3 Ability runtime

Le framework applicatif HarmonyOS repose sur le concept d'Ability (équivalent Activity Android). `ability_lifecycle.cpp` implémente les callbacks (`onStart`, `onActive`, `onInactive`, `onBackground`, `onForeground`, `onStop`). `ability_manager.cpp` gère la loading depuis HAP, le start/terminate, et la ability stack.

9. Module afros-dxvk

Couche de traduction DirectX → Vulkan. Implémente D3D9, D3D11, D3D12, DXGI, un compilateur HLSL → SPIR-V, et un backend Vulkan avec memory allocator, pipeline cache, et presenter swapchain.

9.1 Sous-systèmes

Sous-système	Fichiers	Rôle
src/d3d9/	5	D3D9 factory, device, shader (SM2/3), state, swapchain
src/d3d11/	5	D3D11 device, context (deferred flush), buffer, texture, shader
src/d3d12/	4	D3D12 command list, resource, descriptor heap, device
src/dxgi/	3	DXGI factory, adapter, swapchain
src/hlsl/	3	HLSL compiler (lexer+parser), optimizer (DCE), SPIR-V emitter
src/vulkan/	4	Vulkan device, VMA-style memory allocator, pipeline cache, presenter
src/util/	3	Cache manager (LRU on disk), shader cache (XXH64), perf monitor

9.2 Points d'entrée Wine

DXVK s'exécute comme un ensemble de DLLs Windows chargées par Wine. Les points d'entrée sont exportés en extern "C" pour permettre le chargement via dlopen.

```
// Entry points chargés par Wine extern "C" IDirect3D9* Direct3DCreate9(UINT sdk);
extern "C" HRESULT CreateDXGIFactory(REFIID riid, void **factory); extern "C"
HRESULT D3D11CreateDevice(IDXGIAdapter *adapter, D3D_DRIVER_TYPE type, HMODULE sw,
UINT flags, const D3D_FEATURE_LEVEL *levels, UINT levels_count, UINT sdk,
ID3D11Device **device, D3D_FEATURE_LEVEL *fl, ID3D11DeviceContext **ctx);
```

9.3 Auto-déclaration Vulkan

Faute de Vulkan SDK dans l'environnement de build, DXVK redéclare les prototypes Vulkan minimaux (types Vk*, enums VkFormat, codes HRESULT) au lieu d'inclure <vulkan/vulkan.h>. Cela permet la compilation syntaxique sans dépendance externe, et le code peut être lié à une vraie libvulkan à l'exécution.

10. Modules support

Trois modules support complètent l'écosystème AfriOS : afros-network (stack réseau), afros-storage (gestion stockage), afros-power-management (gestion énergie). Ces modules consomment la HAL et exposent des APIs utilisées par le noyau et l'orchestrateur.

10.1 afros-network

Stack réseau intelligent avec API socket BSD-style. Expose `afros_net_open_socket`, `afros_net_send`, `afros_net_recv`, `afros_net_close_socket`. Implémentation host (64 sockets, `SO_RCVTIMEO`) + stubs freestanding (`AFROS_ERROR_NOT_SUPPORTED`). Consommé par le module DTN `intermittent_net.c` pour le rejeu de paquets.

10.2 afros-storage

Gestion du stockage avec AfrosFS (filesystem journalisé). Expose `storage_init`, `storage_get_ops` (vtable avec `enable_encryption`, `write_file`, `read_file`). Délègue à la HAL `arch_storage_ops` pour les accès blocs.

10.3 afros-power-management

Gestion d'énergie avec états (active/idle/suspend/hibernate), monitoring batterie, et mode économie. Expose `power_manager_init`, `power_set_state`, `power_get_current_state`, `power_is_on_ac_power`, `power_set_performance_mode`. Consommé par `solar_aware.c` et `power_aware_sched.c`.

11. Build, tests et CI

Cette section décrit les modules qui ne sont pas du code applicatif mais qui supportent le build, les tests, et l'automatisation.

11.1 Build system

- CMakeLists.txt racine — options AFROS_PORT, AFROS_BUILD_KERNEL, AFROS_BUILD_FIRMWARE, AFROS_BUILD_COMPAT, AFROS_BUILD_TESTS, AFROS_FREESTANDING.
- cmake/toolchains/ — 4 toolchains cross-compilation : toolchain-arm64.cmake, toolchain-x86_64.cmake, toolchain-riscv.cmake, toolchain-mcu.cmake.
- CMakeLists par module — 10 fichiers CMakeLists.txt modernisés (CMake $\geq$ 3.16, target-based, C++17, OBJC probe pour afros-incompat-engine).

11.2 Tests

- HAL tests — hal_test_runner.c 42 tests via port-host-mock. CTest intégré.
- Compat tests — 18 tests (5 Windows, 4 Android, 4 iOS/macOS, 3 HarmonyOS, 2 Linux baseline). Harness Python compat-test-harness.py.
- Tests Linux baseline — compilés et validés en local (hello-elf, fork-exec).

11.3 CI/CD

- ci-workflows/afrios-ci.yml — workflow GitHub Actions 7 jobs.
- ci-workflows/afrios-release.yml — workflow release sur tags v*.
- scripts/ci-syntax-check.py — 506 lignes, parallel 8 workers, 16 s pour 337 fichiers.
- scripts/ci-artifacts-summary.py — 374 lignes, rapport Markdown + JSON.

11.4 Scripts d'automatisation

Script	Rôle
fetch-deps.sh	Clone shallow 9 deps externes pinnées
check-deps.sh	Vérifie présence + version des deps
update-dep.sh	Met à jour un pin + re-fetch
install-workflows.sh	Bootstrap GitHub workflows
run-hal-tests.sh	Build + exécution tests HAL
setup.sh	Orchestrateur 7 phases one-shot
run-all-tests.sh	6 suites de tests unifiées

12. Annexe — Index des fichiers clés

Cette annexe liste les fichiers les plus importants du dépôt, classés par ordre alphabétique, avec leur module d'appartenance et leur rôle.

Fichier	Module	Rôle
api_version.c	corebridge	Entry points orchestrator (init/run/shutdown)
AppDetect()	corebridge/loader	Détection format par magic bytes
BootManager/*	firmware	Boot multi-OS (Win/macOS/PXE/HTTP)
CMakeLists.txt	tous	Build system CMake modernisé
d3d11_device.cpp	dxvk	ID3D11Device impl
fetch-deps.sh	scripts	Vendorisation 9 deps externes
hal_test_runner.c	afros-core	42 tests HAL (host-mock)
intermittent_net.c	afros-core/network	DTN tolérant aux coupures
kernel_main()	afros-core	Entry point noyau
kprintf.c	afros-core/hal	printf freestanding
MachoParse()	incompat-engine	Parser Mach-O
orchestrator_run_app()	corebridge	Lancement app multi-runtime
Passthrough.c	firmware	Passthrough ETP/Stage-2 hyperviseur
solar_aware.c	afros-core/power	Gestion énergie solaire
tcp_westwood_africa.c	afros-core/network	TCP Westwood+ Africa
vector_table.S	afros-core/ports/port-mcu	Table vecteurs Cortex-M
VfsCreateView()	corebridge/unified_executable	VFS union multi-runtime
WinRuntimeSpawn()	corebridge/runtime_manager	Démarrage wineserver

13. Conclusion

Ce document a présenté les 12 modules qui composent AfriOS, du firmware UEFI aux couches de compatibilité applicative. Chaque module a un périmètre strict, une API publique documentée, et des dépendances claires vers les autres modules. L'architecture en couches permet une évolution indépendante et facilite les tests.

Les modules les plus complexes sont afros-androsandbox (7 332 lignes, 41 fichiers) et afros-incompat-engine (6 978 lignes, 42 fichiers) — ces deux modules wrappent des runtimes entiers (ART et Darling) avec leur propre scheduler, leur propre IPC, et leur propre framework UI. Le module afros-corebridge-core est le cœur fonctionnel qui

orchestre tout cela avec une API gelée v1.0.0.

Pour le détail de l'implémentation de chaque fonction, se reporter au code source documenté (chaque fichier a un header `/** @file */` et chaque fonction a un doc comment). Pour les choix d'architecture, se reporter au document 2/3 Nouvelle architecture. Pour l'historique de l'implémentation, se reporter au document 1/3 Présentation de l'implémentation et au worklog consolidé WORKLOG-COMPLETION.md (3 100+ lignes).